Series temporales y pronóstico

Guía académica 04 · PyTorch Lightning frente a Keras 3 sobre JAX

Juan Sebastián Laverde Chunza

15 de octubre de 2026

Esta guía construye un *pipeline* de pronóstico de series temporales con dos implementaciones. Revisa la división temporal sin fugas de información, la normalización con estadísticas solo del entrenamiento, las ventanas deslizantes, las líneas base ingenuas y las métricas en unidades originales. Primero entrena un pronosticador con PyTorch Lightning. Después entrena el mismo modelo con Keras 3 sobre el *backend* JAX, alimentado por los mismos `DataLoader` de PyTorch, y compara ambos enfoques. Todo el código se ejecutó al generar el documento.

Tabla de contenidos

1 Propósito y mapa de lectura **2**

2 Intuición: predecir el siguiente compás **3**

3 Fundamentos **3**
3.1 De serie a problema supervisado . . . 3
3.2 División temporal y fugas de información . . . 3
3.3 Líneas base . . . 3
3.4 Métricas en unidades originales . . . 4

4 Datos: una serie con ciclos irregulares **4**
4.1 División cronológica y normalización . . . 5

5 Ventanas deslizantes **6**
5.1 Un `Dataset` de ventanas . . . 6

6 PyTorch Lightning **7**
6.1 El `LightningModule` . . . 7
6.2 El `Trainer` . . . 8
6.3 Predicción con el mejor modelo . . . 9

7 Keras 3 sobre el *backend* JAX **9**

8 Evaluación en unidades originales **10**
8.1 Comparación con las líneas base . . . 10
8.2 Pronóstico recursivo a varios meses . . . 11
8.3 Estrategia directa: un modelo que produce doce meses a la vez . . . 12

9 Comparación de enfoques **13**

10 Dimensiones de tensores y flujo de datos 14

11 Hiperparámetros del modelo 15

12 Errores frecuentes 16

13 Ejercicios 17
13.1 Respuestas orientativas 17

14 Glosario 17

15 Conclusiones académicas 18

16 Referencias 19

Procedencia y autoría

Autor de esta guía. Juan Sebastián Laverde Chunza, con apoyo de herramientas de inteligencia artificial generativa. Es material académico complementario y **no** es material oficial del docente ni de la Universidad El Bosque.
Referencia académica. Los temas siguen el notebook `Pipeline__PyTorchL_KerasJAX.ipynb` de la Clase 02 del curso Deep Learning I, del profesor **Héctor Javier Hortúa, PhD** (GitHub · Colab).
Edición pública. El código, los datos, los ejemplos y las explicaciones de esta edición son propios e independientes: no reproducen el código ni el texto del notebook del docente. El notebook de la clase usa la serie mensual de manchas solares. Esta guía usa una serie sintética con un ciclo de unos once años, para funcionar sin archivos externos. Citar el notebook como referencia no equivale a una licencia ni a una autorización sobre el material original.

1. Propósito y mapa de lectura

Pronosticar es estimar valores futuros de una serie a partir de su pasado: la demanda eléctrica de mañana, las ventas del próximo mes o la actividad solar del siguiente ciclo. A diferencia de los problemas de las guías anteriores, **el orden de los datos importa**. Mezclar observaciones de distintos momentos destruye la información temporal y produce evaluaciones engañosas.

Bloque	Contenido
§3 Fundamentos	Encuadre supervisado, ventanas, fugas de información, líneas base y métricas
§4 Datos	Serie sintética, división temporal y normalización
§5 Ventanas	Un `Dataset` de ventanas deslizantes y sus `DataLoader`
§6 Lightning	`LightningModule`, `Trainer`, registro, *checkpoints* y programador de tasa
§7 Keras/JAX	El mismo modelo en Keras 3 con *backend* JAX, entrenado con los `DataLoader` de PyTorch
§8 Evaluación	Métricas en unidades originales, comparación con líneas base y pronóstico recursivo

2. Intuición: predecir el siguiente compás

Quien escucha una canción conocida puede anticipar la nota siguiente porque recuerda las anteriores. Un pronosticador hace lo mismo con una ventana de observaciones recientes: aprende qué suele venir después de un patrón. Hay dos reglas de honestidad:

- No se puede usar el futuro para predecir el pasado. La evaluación debe simular que solo se conoce lo ocurrido hasta cada momento.
- Antes de celebrar un modelo, hay que compararlo con reglas muy simples, como «mañana será igual que hoy». Si no las supera, no aporta valor.

3. Fundamentos

3.1. De serie a problema supervisado

Dada una serie $y_1, y_2, \dots, y_T$, una **ventana** de longitud w y un **horizonte** h, cada ejemplo de entrenamiento es

$$\mathbf{x}_t = (y_{t-w+1}, \dots, y_t), \qquad \mathbf{y}_t = (y_{t+1}, \dots, y_{t+h}).$$

El modelo aprende $f : \mathbb{R}^w \to \mathbb{R}^h$. Con T observaciones se obtienen $T - w - h + 1$ ejemplos.

Ejemplo resuelto. Para la serie $(3, 5, 4, 6, 7, 9)$, con $w = 3$ y $h = 1$, hay $6 - 3 - 1 + 1 = 3$ ejemplos: $(3, 5, 4) \to 6$, $(5, 4, 6) \to 7$ y $(4, 6, 7) \to 9$. Con $h = 2$ habría dos: $(3, 5, 4) \to (6, 7)$ y $(5, 4, 6) \to (7, 9)$.

3.2. División temporal y fugas de información

En series temporales la división se hace **por bloques cronológicos**: entrenamiento, después validación, después prueba. Una partición aleatoria pondría en entrenamiento observaciones posteriores a las de prueba. El modelo «vería el futuro» y la métrica sería optimista.

La misma regla vale para la **normalización**:

$$\tilde{y}_t = \frac{y_t - \mu_{\text{train}}}{\sigma_{\text{train}}}.$$

La media y la desviación estándar se calculan solo con el bloque de entrenamiento. Calcularlas con toda la serie filtraría el nivel y la variabilidad del futuro.

3.3. Líneas base

Línea base	Pronóstico de y_{t+1}	Qué captura
Persistencia	y_t	La inercia de la serie
Media móvil	$\frac{1}{k}\sum_{i=0}^{k-1} y_{t-i}$	El nivel reciente, suavizado
Estacional ingenua	y_{t+1-s}, con periodo s	Un ciclo que se repite

3.4. Métricas en unidades originales

El modelo se entrena con la serie normalizada, pero las métricas se reportan después de invertir la normalización ($y = \tilde{y}\,\sigma_{\text{train}} + \mu_{\text{train}}$), para que tengan significado práctico:

$$\text{MAE} = \frac{1}{n}\sum |y_i - \hat{y}_i|, \qquad \text{RMSE} = \sqrt{\frac{1}{n}\sum (y_i - \hat{y}_i)^2}.$$

La RMSE penaliza más los errores grandes. Si la RMSE es mucho mayor que el MAE, hay errores grandes ocasionales.

```
import os
os.environ["KERAS_BACKEND"] = "jax"      # antes de cualquier import de keras
os.environ["XLA_PYTHON_CLIENT_PREALLOCATE"] = "false"

import logging, warnings, tempfile, pathlib
warnings.filterwarnings("ignore")
logging.getLogger("lightning.pytorch").setLevel(logging.ERROR)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
from torch import nn
from torch.utils.data import Dataset, DataLoader
import lightning as L

print("PyTorch", torch.__version__, "| Lightning", L.__version__)
```

```
PyTorch 2.14.1+cpu | Lightning 2.6.6
```

4. Datos: una serie con ciclos irregulares

La serie sintética imita un fenómeno cíclico no negativo. Tiene ciclos de unos 132 meses (once años) cuya duración y amplitud varían de un ciclo a otro, y ruido proporcional al nivel. Esa irregularidad hace que la línea base estacional sea imperfecta y que valga la pena aprender.

```
def cyclic_activity(n_months=2400, mean_period=132, seed=17):
    gen = np.random.default_rng(seed)
    values, t = [], 0
    while len(values) < n_months:
        period = int(gen.normal(mean_period, 12))
        peak = gen.gamma(shape=6.0, scale=18.0)
        phase = np.linspace(0, np.pi, period, endpoint=False)
        shape = np.sin(phase) ** 1.6
        values.extend(peak * shape)
    level = np.array(values[:n_months])
    noise = gen.normal(0.0, 1.0, n_months) * (4.0 + 0.15 * level)
    return np.clip(level + noise, 0.0, None).astype(np.float32)

activity = cyclic_activity()
months = pd.period_range("1800-01", periods=len(activity), freq="M")
print("observaciones:", len(activity), "| desde", months[0],
      "hasta", months[-1])
```

```
print(f"media={activity.mean():.1f}  desv={activity.std():.1f}  "
      f"max={activity.max():.1f}")
```

```
observaciones: 2400 | desde 1800-01 hasta 1999-12
media=56.9  desv=46.7  max=296.1
```

4.1. División cronológica y normalización

```
n = len(activity)
cut_a, cut_b = int(0.70 * n), int(0.85 * n)
blocks = {"entrenamiento": activity[:cut_a],
          "validacion": activity[cut_a:cut_b],
          "prueba": activity[cut_b:]}
for name, block in blocks.items():
    print(f"{name:<14} {len(block):>5} meses")

mu, sigma = blocks["entrenamiento"].mean(), blocks["entrenamiento"].std()
scaled = (activity - mu) / sigma
print(f"estadisticas de entrenamiento: mu={mu:.2f}  sigma={sigma:.2f}")
```

```
entrenamiento   1680 meses
validacion       360 meses
prueba           360 meses
estadisticas de entrenamiento: mu=55.58  sigma=47.92
```

```
fig, ax = plt.subplots(figsize=(7.2, 2.7))
x_axis = np.arange(n) / 12 + 1800
ax.plot(x_axis[:cut_a], activity[:cut_a], lw=0.6, label="entrenamiento")
ax.plot(x_axis[cut_a:cut_b], activity[cut_a:cut_b], lw=0.6,
        label="validación")
ax.plot(x_axis[cut_b:], activity[cut_b:], lw=0.6, label="prueba")
ax.set_xlabel("año"); ax.set_ylabel("actividad"); ax.legend(ncol=3)
plt.tight_layout(); plt.show()
```

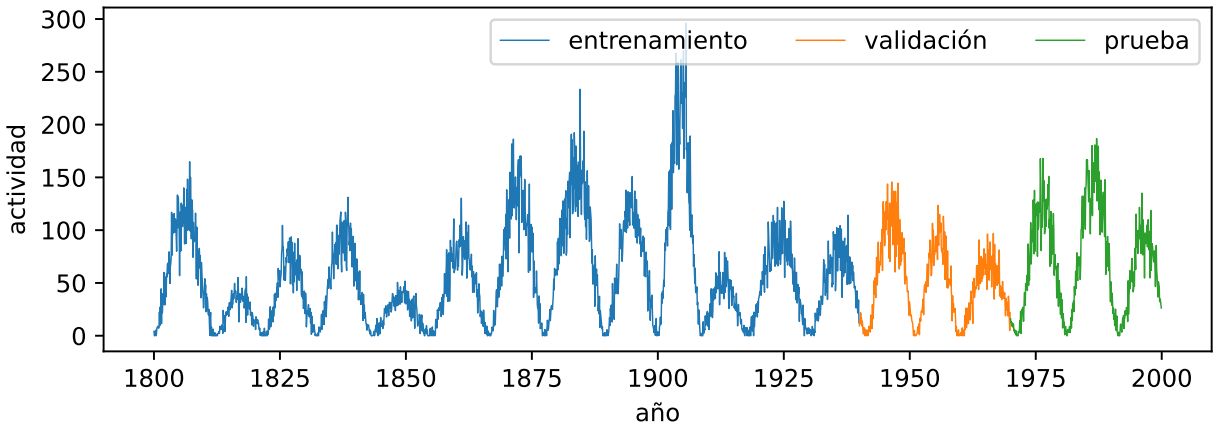


Figura 1: Serie sintética con los tres bloques cronológicos.

5. Ventanas deslizantes

5.1. Un `Dataset` de ventanas

La clase recibe la serie completa normalizada y el rango de **índices objetivo** que le corresponde. Así las ventanas de validación pueden usar como contexto los últimos meses de entrenamiento. No es una fuga, porque ese pasado ya se conocía en el momento del pronóstico. Lo que nunca ocurre es que un objetivo de entrenamiento caiga dentro de validación o prueba.

```
class SlidingWindows(Dataset):
    """Pares (ultimos `window` valores, siguientes `horizon` valores)."""

    def __init__(self, series, first_target, last_target, window, horizon=1):
        self.series = torch.as_tensor(series, dtype=torch.float32)
        self.window, self.horizon = window, horizon
        start = max(first_target, window)
        stop = last_target - horizon + 1
        self.targets = torch.arange(start, stop)

    def __len__(self):
        return len(self.targets)

    def __getitem__(self, i):
        t = int(self.targets[i])
        context = self.series[t - self.window:t]
        future = self.series[t:t + self.horizon]
        return context, future

toy = SlidingWindows(np.array([3, 5, 4, 6, 7, 9]), 0, 6, window=3)
for ctx, fut in toy:
    print(ctx.numpy(), "->", fut.numpy())
```

```
[3. 5. 4.] -> [6.]
[5. 4. 6.] -> [7.]
[4. 6. 7.] -> [9.]
```

La salida reproduce el ejemplo resuelto de la §3.1. Ahora se crean los conjuntos reales con una ventana de 48 meses.

```
WINDOW, HORIZON, BATCH = 48, 1, 64
fit_ds = SlidingWindows(scaled, 0, cut_a, WINDOW, HORIZON)
check_ds = SlidingWindows(scaled, cut_a, cut_b, WINDOW, HORIZON)
exam_ds = SlidingWindows(scaled, cut_b, n, WINDOW, HORIZON)

fit_loader = DataLoader(fit_ds, batch_size=BATCH, shuffle=True,
                        generator=torch.Generator().manual_seed(0))
check_loader = DataLoader(check_ds, batch_size=256)
exam_loader = DataLoader(exam_ds, batch_size=256)
print("ventanas:", len(fit_ds), len(check_ds), len(exam_ds))
xb, yb = next(iter(fit_loader))
print("lote:", tuple(xb.shape), "->", tuple(yb.shape))
```

```
ventanas: 1632 360 360
lote: (64, 48) -> (64, 1)
```

Dentro del entrenamiento sí se pueden **mezclar las ventanas** (`shuffle=True`). Cada ventana ya contiene su propio orden temporal, y mezclarlas solo cambia el orden en que el optimizador las ve. Lo que no se puede mezclar es la asignación de meses a los bloques.

6. PyTorch Lightning

6.1. El `LightningModule`

El modelo es un perceptrón multicapa que mira la ventana como un vector de 48 valores. El módulo declara el paso de entrenamiento, el de validación y el de predicción. También declara el optimizador, con un programador `ReduceLROnPlateau` que vigila la pérdida de validación.

```
def window_mlp(window, horizon, width=64):
    return nn.Sequential(
        nn.Linear(window, width), nn.GELU(),
        nn.Linear(width, width), nn.GELU(),
        nn.Linear(width, horizon))

class Forecaster(L.LightningModule):
    def __init__(self, window=WINDOW, horizon=HORIZON, width=64, lr=2e-3):
        super().__init__()
        self.save_hyperparameters()
        self.net = window_mlp(window, horizon, width)

    def forward(self, context):
        return self.net(context)

    def _step(self, batch, stage):
        context, future = batch
        pred = self(context)
        loss = nn.functional.mse_loss(pred, future)
        mae = (pred - future).abs().mean()
        self.log(f"{stage}_loss", loss, on_epoch=True, on_step=False)
        self.log(f"{stage}_mae", mae, on_epoch=True, on_step=False)
        return loss

    def training_step(self, batch, batch_idx):
        return self._step(batch, "train")

    def validation_step(self, batch, batch_idx):
        self._step(batch, "val")

    def predict_step(self, batch, batch_idx):
        return self(batch[0])

    def configure_optimizers(self):
        opt = torch.optim.Adam(self.parameters(), lr=self.hparams.lr)
        sched = torch.optim.lr_scheduler.ReduceLROnPlateau(
            opt, factor=0.5, patience=4)
        return {"optimizer": opt,
                "lr_scheduler": {"scheduler": sched, "monitor": "val_loss"}}
```

6.2. El `Trainer`

`CSVLogger` escribe las métricas de cada época en un archivo. `TensorBoardLogger` haría lo mismo para TensorBoard. `ModelCheckpoint` guarda el mejor modelo y `EarlyStopping` corta el entrenamiento cuando la validación se estanca.

```
from lightning.pytorch.callbacks import (EarlyStopping, ModelCheckpoint,
                                         LearningRateMonitor)
from lightning.pytorch.loggers import CSVLogger

L.seed_everything(5, verbose=False)
out_dir = pathlib.Path(tempfile.mkdtemp(prefix="guia04_"))
csv_logger = CSVLogger(out_dir, name="lightning")
keeper = ModelCheckpoint(monitor="val_loss", mode="min",
                         filename="pronostico-{epoch:03d}")
trainer = L.Trainer(
    max_epochs=120, accelerator="cpu", deterministic=True,
    logger=csv_logger, enable_progress_bar=False, enable_model_summary=False,
    callbacks=[keeper, EarlyStopping("val_loss", patience=12),
               LearningRateMonitor("epoch")])
forecaster = Forecaster()
trainer.fit(forecaster, fit_loader, check_loader)
print("epocas:", trainer.current_epoch,
      "| mejor val_loss:", round(float(keeper.best_model_score), 4))
print("checkpoint:", pathlib.Path(keeper.best_model_path).name)
```

```
epocas: 40 | mejor val_loss: 0.0759
checkpoint: pronostico-epoch=027.ckpt
```

```
log = pd.read_csv(pathlib.Path(csv_logger.log_dir) / "metrics.csv")
per_epoch = log.groupby("epoch").mean(numeric_only=True)
fig, axes = plt.subplots(1, 2, figsize=(7.2, 2.7))
axes[0].plot(per_epoch.index, per_epoch["train_loss"], label="entrenamiento")
axes[0].plot(per_epoch.index, per_epoch["val_loss"], label="validación")
axes[0].set_yscale("log"); axes[0].set_xlabel("época")
axes[0].set_ylabel("MSE (escala normalizada)"); axes[0].legend()
lr_col = [c for c in per_epoch.columns if c.startswith("lr")][0]
axes[1].plot(per_epoch.index, per_epoch[lr_col])
axes[1].set_yscale("log"); axes[1].set_xlabel("época")
axes[1].set_ylabel("tasa de aprendizaje")
plt.tight_layout(); plt.show()
```

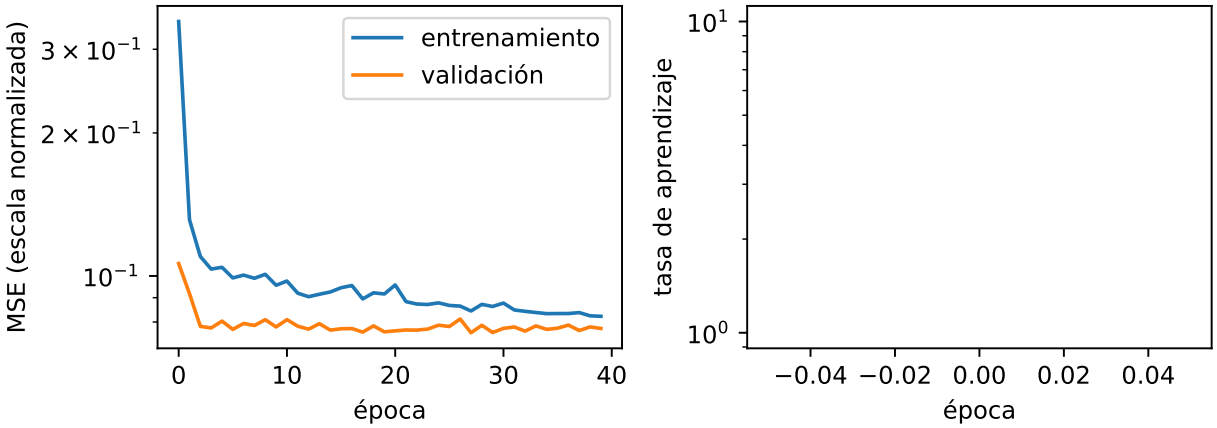


Figura 2: Registro del CSVLogger: pérdidas por época y tasa de aprendizaje.

6.3. Predicción con el mejor modelo

```
best_lit = Forecaster.load_from_checkpoint(keeper.best_model_path)
pred_scaled = torch.cat(trainer.predict(best_lit, exam_loader)).numpy()[:, 0]
truth_scaled = torch.stack([fut for _, fut in exam_ds]).numpy()[:, 0]

def to_units(z):
    return z * sigma + mu

lit_pred, truth = to_units(pred_scaled), to_units(truth_scaled)
print("predicciones de prueba:", lit_pred.shape)
```

```
predicciones de prueba: (360,)
```

7. Keras 3 sobre el *backend* JAX

Keras 3 puede ejecutar el mismo modelo sobre TensorFlow, PyTorch o JAX. El *backend* se fijó como `"jax"` en la primera celda, antes de importar Keras. Con JAX, los pasos de entrenamiento se compilan con XLA. Keras acepta directamente los `DataLoader` de PyTorch definidos antes, de modo que ambos modelos se entrenan con **exactamente los mismos lotes**.

```
import keras
import jax
keras.utils.set_random_seed(5)
print("backend:", keras.backend.backend(), "| JAX", jax.__version__,
      "| dispositivo:", jax.devices()[0].platform)

keras_forecaster = keras.Sequential([
    keras.Input(shape=(WINDOW,)),
    keras.layers.Dense(64, activation="gelu"),
    keras.layers.Dense(64, activation="gelu"),
    keras.layers.Dense(HORIZON),
], name="pronosticador_keras")
keras_forecaster.compile(optimizer=keras.optimizers.Adam(2e-3), loss="mse",
                         metrics=[keras.metrics.MeanAbsoluteError(name="mae")])
keras_hist = keras_forecaster.fit(
```

```
    fit_loader, validation_data=check_loader, epochs=120, verbose=0,
    callbacks=[keras.callbacks.EarlyStopping(patience=12,
                                             restore_best_weights=True),
               keras.callbacks.ReduceLROnPlateau(factor=0.5, patience=4)])
print("epocas:", len(keras_hist.history["loss"]),
      "| mejor val_loss:", round(min(keras_hist.history["val_loss"]), 4))
keras_pred = to_units(keras_forecaster.predict(exam_loader, verbose=0)[:, 0])
```

```
backend: jax | JAX 0.11.2 | dispositivo: cpu
epocas: 19 | mejor val_loss: 0.0791
```

Las dos arquitecturas son idénticas: $48 \to 64 \to 64 \to 1$ con GELU, y $(48 \cdot 64 + 64) + (64 \cdot 64 + 64) + (64 + 1) = 7\,361$ parámetros. Lo que cambia es el motor: el autograd dinámico de PyTorch en un caso y la compilación XLA de JAX en el otro.

8. Evaluación en unidades originales

8.1. Comparación con las líneas base

```
targets = exam_ds.targets.numpy()
persistence = activity[targets - 1]
moving_avg = np.array([activity[t - 6:t].mean() for t in targets])
seasonal = activity[targets - 132]

def mae(a, b):
    return float(np.mean(np.abs(a - b)))

def rmse(a, b):
    return float(np.sqrt(np.mean((a - b) ** 2)))

table = pd.DataFrame({
    "MAE": [mae(truth, p) for p in
            (persistence, moving_avg, seasonal, lit_pred, keras_pred)],
    "RMSE": [rmse(truth, p) for p in
             (persistence, moving_avg, seasonal, lit_pred, keras_pred)],
}, index=["persistencia", "media movil (6)", "estacional (132)",
          "Lightning (MLP)", "Keras/JAX (MLP)"]).round(2)
print(table.to_string())
```

```
                   MAE   RMSE
persistencia     15.73  21.49
media movil (6)  13.88  18.36
estacional (132) 31.27  41.00
Lightning (MLP)  12.49  17.37
Keras/JAX (MLP)  12.28  17.11
```

> Cómo leer la tabla
>
> La línea base estacional es la peor. Los ciclos varían en duración y amplitud, y el mes equivalente del ciclo anterior es una mala guía. La persistencia y la media móvil son difíciles de batir en un horizonte de un mes, porque la serie cambia poco de un mes al siguiente. Un

modelo útil debe mejorar esas cifras. Si los dos MLP quedan cerca entre sí, se confirma que el *framework* no cambia lo que se aprende: cambia cómo se escribe y cómo se ejecuta.

```
show = slice(-180, None)
fig, ax = plt.subplots(figsize=(7.2, 2.8))
years = targets[show] / 12 + 1800
ax.plot(years, truth[show], color="black", lw=1.0, label="real")
ax.plot(years, lit_pred[show], lw=0.9, label="Lightning")
ax.plot(years, keras_pred[show], lw=0.9, ls="--", label="Keras/JAX")
ax.set_xlabel("año"); ax.set_ylabel("actividad"); ax.legend(ncol=3)
plt.tight_layout(); plt.show()
```

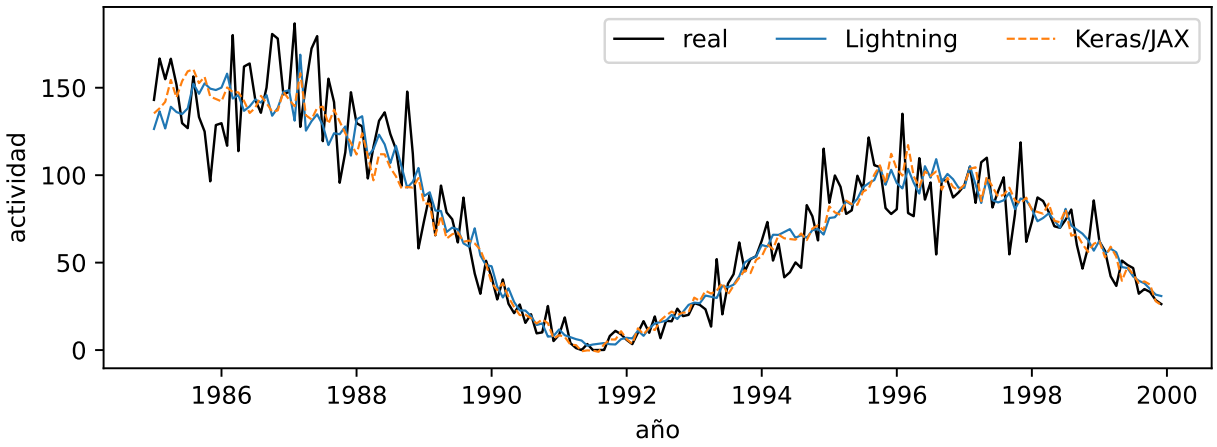


Figura 3: Últimos 15 años de prueba: valores reales y pronósticos a un mes.

8.2. Pronóstico recursivo a varios meses

Para pronosticar más de un paso con un modelo de horizonte 1, se puede usar la **estrategia recursiva**: se predice un mes, se agrega la predicción a la ventana como si fuera un dato y se repite. Los errores se acumulan, porque cada paso usa predicciones anteriores.

```
@torch.inference_mode()
def recursive_forecast(model, history_scaled, steps):
    model.eval()
    window = torch.as_tensor(history_scaled[-WINDOW:], dtype=torch.float32)
    outputs = []
    for _ in range(steps):
        nxt = model(window.unsqueeze(0))[0, 0]
        outputs.append(float(nxt))
        window = torch.cat([window[1:], nxt.unsqueeze(0)])
    return to_units(np.array(outputs))

STEPS = 36
multi = recursive_forecast(best_lit, scaled[:cut_b], STEPS)
real = activity[cut_b:cut_b + STEPS]
for k in (1, 6, 12, 24, 36):
    err = np.abs(multi[:k] - real[:k]).mean()
    print(f"MAE de los primeros {k:>2} meses: {err:6.2f}")
```

```
MAE de los primeros  1 meses:   2.11
MAE de los primeros  6 meses:   2.10
MAE de los primeros 12 meses:   2.09
MAE de los primeros 24 meses:   2.75
MAE de los primeros 36 meses:   6.40
```

En esta ejecución, el MAE acumulado se mantiene estable durante el primer año y luego crece con rapidez: cada paso adicional se apoya en más predicciones propias. Estas cifras cubren solo los primeros 36 meses del bloque de prueba, así que no son comparables con la tabla anterior, que promedia todo el bloque.

8.3. Estrategia directa: un modelo que produce doce meses a la vez

La alternativa es entrenar el modelo con $h = 12$: la última capa tiene 12 salidas y la pérdida promedia el error de los 12 meses. Para compararla con la estrategia recursiva de forma justa, se pronostican los 12 meses siguientes **desde cada origen** del bloque de prueba y se promedia el error por adelanto (*lead*).

```
LEAD = 12
fit12 = SlidingWindows(scaled, 0, cut_a, WINDOW, LEAD)
check12 = SlidingWindows(scaled, cut_a, cut_b, WINDOW, LEAD)
exam12 = SlidingWindows(scaled, cut_b, n, WINDOW, LEAD)

L.seed_everything(5, verbose=False)
direct_keeper = ModelCheckpoint(monitor="val_loss", mode="min")
direct_trainer = L.Trainer(
    max_epochs=120, accelerator="cpu", deterministic=True, logger=False,
    enable_progress_bar=False, enable_model_summary=False,
    callbacks=[direct_keeper, EarlyStopping("val_loss", patience=12)])
direct_trainer.fit(
    Forecaster(horizon=LEAD),
    DataLoader(fit12, batch_size=BATCH, shuffle=True,
               generator=torch.Generator().manual_seed(0)),
    DataLoader(check12, batch_size=256))
direct = Forecaster.load_from_checkpoint(direct_keeper.best_model_path)
direct.eval()

contexts = torch.stack([ctx for ctx, _ in exam12])
futures = to_units(torch.stack([fut for _, fut in exam12]).numpy())
with torch.inference_mode():
    direct_pred = to_units(direct(contexts).numpy())
    window = contexts.clone()
    rec_steps = []
    for _ in range(LEAD):
        nxt = best_lit(window)
        rec_steps.append(nxt)
        window = torch.cat([window[:, 1:], nxt], dim=1)
    recursive_pred = to_units(torch.cat(rec_steps, dim=1).numpy())

mae_direct = np.abs(direct_pred - futures).mean(axis=0)
mae_recursive = np.abs(recursive_pred - futures).mean(axis=0)
print("origenes evaluados:", len(contexts))
for lead in (1, 3, 6, 9, 12):
    print(f"adelanto {lead:>2}: MAE recursiva={mae_recursive[lead - 1]:6.2f}"
          f"  directa={mae_direct[lead - 1]:6.2f}")
```

```
origenes evaluados: 349
adelanto  1: MAE recursiva= 12.70  directa= 12.16
adelanto  3: MAE recursiva= 12.97  directa= 12.48
adelanto  6: MAE recursiva= 13.24  directa= 12.94
adelanto  9: MAE recursiva= 13.68  directa= 13.40
adelanto 12: MAE recursiva= 14.23  directa= 13.93
```

```
fig, ax = plt.subplots(figsize=(5.6, 2.8))
leads = np.arange(1, LEAD + 1)
ax.plot(leads, mae_recursive, marker="o", ms=3, label="recursiva (h=1)")
ax.plot(leads, mae_direct, marker="s", ms=3, label="directa (h=12)")
ax.set_xlabel("adelanto (meses)"); ax.set_ylabel("MAE")
ax.legend(); plt.tight_layout(); plt.show()
```

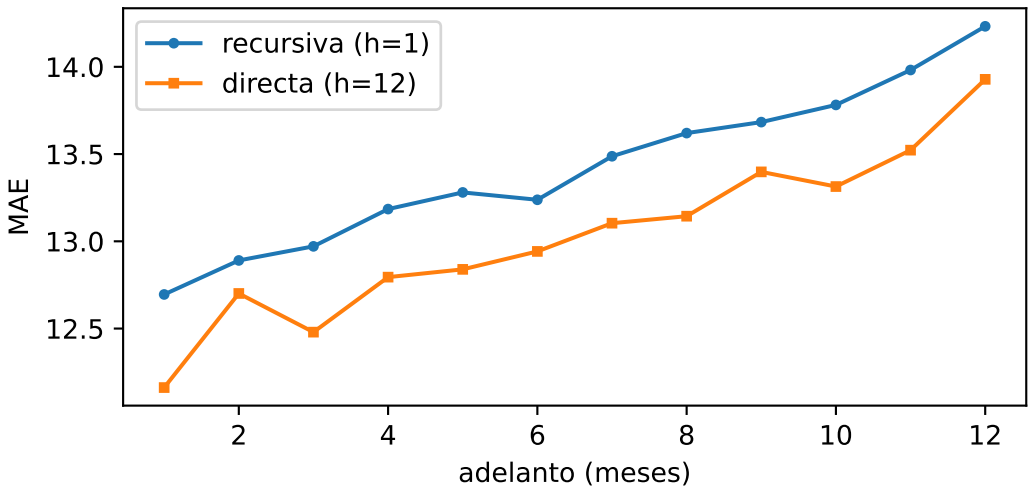


Figura 4: MAE en prueba según el adelanto del pronóstico, promediado sobre todos los orígenes.

En esta ejecución, el error de ambas estrategias crece de forma moderada con el adelanto, y la directa es ligeramente mejor en todos los adelantos. Con este nivel de ruido y un horizonte de 12 meses, la acumulación de errores de la estrategia recursiva es pequeña. En horizontes más largos o en series más suaves la diferencia puede ser mucho mayor. La directa evita acumular errores, aunque cada salida recibe menos señal durante el entrenamiento. Cuál gana depende de la serie y del horizonte, y una curva como la de la figura es la forma correcta de decidirlo.

El MAE a un mes de la estrategia recursiva difiere ligeramente del de la tabla de la §8.1, porque aquí solo se usan los 349 orígenes que tienen 12 meses de futuro disponibles.

9. Comparación de enfoques

Aspecto	PyTorch Lightning	Keras 3 sobre JAX
Definición del modelo	`nn.Module` dentro de un `LightningModule`	`Sequential`, funcional o subclase
Bucle de entrenamiento	`Trainer.fit()`	`model.fit()`

Aspecto	PyTorch Lightning	Keras 3 sobre JAX
Registro de métricas	`self.log()` y *loggers* (CSV, TensorBoard, etc.)	`History` y *callbacks* (`CSVLogger`, `TensorBoard`)
Programador de tasa	Se devuelve en `configure_optimizers`	*Callback* `ReduceLROnPlateau`
Ejecución	Autograd dinámico de PyTorch	Funciones puras compiladas con XLA
Datos	`DataLoader`	`DataLoader`, `tf.data`, arreglos NumPy
Reproducibilidad	`seed_everything`, `deterministic=True`	`keras.utils.set_random_seed`

La elección depende sobre todo del ecosistema: las bibliotecas que ya se usan, la infraestructura de despliegue y la familiaridad del equipo. La metodología no cambia: división temporal, normalización sin fugas, líneas base y métricas en unidades originales.

10. Dimensiones de tensores y flujo de datos

B es el tamaño del lote. Hay 1 632 ventanas de entrenamiento y `BATCH = 64`, así que $1\,632 = 25 \cdot 64 + 32$: 26 lotes por época, y el último tiene $B = 32$. Validación y prueba tienen 360 ventanas cada una y usan `batch_size=256`, lo que da dos lotes de 256 y 104.

Pronosticador a un mes (Lightning y Keras/JAX, la misma arquitectura):

Etapa	Shape	Significado de cada dimensión	Parámetros
Serie normalizada	$(2\,400,)$	Un valor por mes	—
Ventana de entrada	$(B, 48)$	B ventanas × 48 meses de contexto	—
`Linear(48, 64)` + GELU	$(B, 64)$	64 rasgos ocultos por ventana	$48 \cdot 64 + 64 = 3\,136$
`Linear(64, 64)` + GELU	$(B, 64)$	64 rasgos ocultos por ventana	$64 \cdot 64 + 64 = 4\,160$
`Linear(64, 1)`	$(B, 1)$	Pronóstico del mes siguiente	$64 + 1 = 65$
Objetivo	$(B, 1)$	Valor real del mes siguiente	—
Pérdida MSE	$()$	Promedio sobre las B ventanas	—

Estrategia directa (§8.3): la capa final pasa a `Linear(64, 12)` y produce $(B, 12)$, un valor por cada mes del horizonte. Tiene $64 \cdot 12 + 12 = 780$ parámetros, y el total pasa de 7 361 a $3\,136 + 4\,160 + 780 = 8\,076$. Los objetivos también tienen forma $(B, 12)$.

Estrategia recursiva por lotes (§8.3): las 349 ventanas de prueba forman un tensor $(349, 48)$. En cada paso, el modelo produce $(349, 1)$. La ventana se desplaza: se descarta la primera columna y se agrega la predicción, con lo que vuelve a tener forma $(349, 48)$. Tras 12 pasos, las predicciones concatenadas forman $(349, 12)$, la misma forma que las de la estrategia directa, y por eso se pueden comparar columna a columna.

Ejemplo de interpretación. Un lote de entrenamiento $(64, 48)$ contiene $64 \cdot 48 = 3\,072$ valores de la serie. Muchos se repiten, porque ventanas consecutivas comparten 47 meses. La primera capa calcula $(64, 48) \times (48, 64) = (64, 64)$. El MAE por adelanto se obtiene promediando `abs(pred - real)` de forma $(349, 12)$ sobre el eje 0, lo que da un vector $(12,)$.

```
xb, yb = next(iter(fit_loader))
print("lotes:", len(fit_loader), len(check_loader), len(exam_loader),
      "| ultimo de entrenamiento:", len(fit_ds) - 64 * (len(fit_loader) - 1))
print(f"{'etapa':<22}{'shape':>12}{'parametros':>12}")
print(f"{'ventana':<22}{str(tuple(xb.shape)):>12}{'-':>12}")
h_t = xb
with torch.no_grad():
    for layer in best_lit.net:
        h_t = layer(h_t)
        if isinstance(layer, nn.Linear):
            n_p = sum(p.numel() for p in layer.parameters())
            print(f"{'Linear ' + str(tuple(layer.weight.shape)):<22}"
                  f"{str(tuple(h_t.shape)):>12}{n_p:>12}")
    direct_out = direct(xb)
print(f"{'objetivo':<22}{str(tuple(yb.shape)):>12}")
print(f"{'directa (h=12)':<22}{str(tuple(direct_out.shape)):>12}"
      f"{sum(p.numel() for p in direct.parameters()):>12}")
print("Keras/JAX:", keras_forecaster.count_params(), "parametros |",
      "contextos de prueba:", tuple(contexts.shape),
      "| MAE por adelanto:", mae_direct.shape)
```

```
lotes: 26 2 2 | ultimo de entrenamiento: 32
etapa                         shape  parametros
ventana                   (64, 48)           -
Linear (64, 48)           (64, 64)        3136
Linear (64, 64)           (64, 64)        4160
Linear (1, 64)             (64, 1)          65
objetivo                   (64, 1)
directa (h=12)            (64, 12)        8076
Keras/JAX: 7361 parametros | contextos de prueba: (349, 48) | MAE por adelanto: (12,)
```

11. Hiperparámetros del modelo

Hiperparámetro	Valor usado	Función	Al aumentarlo / disminuirlo
Longitud de la serie	2 400 meses	Cantidad de historia	Más historia: más ventanas de entrenamiento
Periodo medio del ciclo	132 meses (desviación 12)	Estructura de la serie sintética	Ciclos más irregulares hacen menos útil la línea base estacional
Partición	70 / 15 / 15 % cronológica	Ajuste, selección y prueba sin fugas	Más prueba da una estimación más estable, con menos datos para entrenar
Ventana (`WINDOW`)	48 meses	Contexto que ve el modelo	Mayor: más contexto (un ciclo entero con 132) pero menos ventanas y más parámetros. Menor: puede no reconocer la fase del ciclo

Hiperparámetro	Valor usado	Función	Al aumentarlo / disminuirlo
Horizonte	1 mes (principal); 12 (estrategia directa)	Pasos pronosticados a la vez	Mayor: tarea más difícil, con errores que crecen con el adelanto
Pasos recursivos	36 meses	Alcance del pronóstico encadenado	Mayor: más acumulación de error
Tamaño de lote	64 (entrenamiento), 256 (evaluación)	Ventanas por paso	Mayor: gradiente más estable y menos pasos por época
Anchura y activación	64 unidades, GELU, 2 capas ocultas	Capacidad del MLP	Más capacidad con la misma serie aumenta el riesgo de sobreajuste
Tasa de aprendizaje	$2 \cdot 10^{-3}$ (Adam)	Tamaño del paso	Muy alta: inestabilidad. Muy baja: convergencia lenta
Programador de tasa	`ReduceLROnPlateau`: `factor=0.5`, `patience=4`	Afina la tasa al estancarse	Más paciencia: menos recortes
`EarlyStopping`	`patience=12`	Corta si la validación no mejora	Mayor: más épocas y más riesgo de sobreajuste
Épocas máximas	120	Tope del entrenamiento	Con *early stopping*, rara vez se alcanza
Líneas base	Media móvil de 6 meses; estacional de 132	Referencias mínimas	Una media más larga suaviza más y reacciona más tarde
Semillas	17 (serie), 5 (modelos), 0 (orden de lotes)	Reproducibilidad	Cambian el resultado concreto, no el método

En términos de aprendizaje, la ventana y el horizonte definen **qué problema** se resuelve. La tasa, el lote y el programador de tasa gobiernan la **convergencia**. La capacidad, las épocas y *patience* gobiernan la **generalización** a periodos futuros.

12. Errores frecuentes

1. **Partición aleatoria de una serie.** Infla las métricas porque el modelo aprende del futuro.
2. **Normalizar con toda la serie.** Filtra la media y la varianza del periodo de prueba.
3. **No comparar con líneas base.** Un RMSE «bajo» puede ser peor que la persistencia.
4. **Reportar métricas en escala normalizada.** No tienen significado práctico; hay que invertir la transformación.
5. **Fijar `KERAS_BACKEND` tarde.** Si Keras ya se importó, la variable no tiene efecto.
6. **Rutas absolutas a los datos.** Un notebook que lee `C:/Users/<usuario>/...` solo funciona en el equipo de quien lo escribió. Conviene usar rutas relativas, descargas desde una URL o datos generados, como en esta guía.
7. **Ignorar el contexto en los bordes.** Si cada bloque se convierte en ventanas por separado, se pierden los primeros `window` objetivos de validación y de prueba.

13. Ejercicios

1. **Ventanas a mano.** Para la serie $(2, 4, 6, 8, 10, 12, 14)$ con $w = 4$ y $h = 2$, ¿cuántos ejemplos hay? Escríbelos.
2. **Longitud de ventana.** Repite el entrenamiento con `WINDOW` igual a 12, 48 y 132. ¿Qué ocurre con el MAE y por qué?
3. **Fuga deliberada.** Normaliza con la media y la desviación de **toda** la serie y vuelve a entrenar. ¿Cambian las métricas? ¿Por qué el cambio es pequeño aquí y puede ser grande en series con tendencia?
4. **Estrategia directa en Keras.** Reproduce la estrategia directa de la §8.3 con Keras sobre JAX y compara su curva de MAE por adelanto con la de Lightning.
5. **Otra arquitectura.** Sustituye el MLP por una capa `nn.LSTM` (en Lightning) y `keras.layers.LSTM` (en Keras). Recuerda que las capas recurrentes esperan entradas `(lote, tiempo, rasgos)`.
6. **Otro *backend*.** Cambia `KERAS_BACKEND` a `"torch"` y reinicia el *kernel*. ¿Cambian los resultados? ¿Y el tiempo por época?

13.1. Respuestas orientativas

1. $7 - 4 - 2 + 1 = 2$ ejemplos: $(2, 4, 6, 8) \to (10, 12)$ y $(4, 6, 8, 10) \to (12, 14)$.
2. Con 12 meses el modelo ve solo una fracción del ciclo y le cuesta saber en qué fase está. Con 132 ve un ciclo completo, pero tiene más parámetros y menos ejemplos de entrenamiento. Suele haber un punto intermedio.
3. Los cambios son pequeños porque esta serie es aproximadamente estacionaria: su nivel medio no cambia mucho. En una serie con tendencia, la media global mezcla niveles del futuro y el error se subestima.
4. Basta con cambiar la capa final a `Dense(12)` y usar los `DataLoader` de horizonte 12. Las curvas deberían ser parecidas a las de Lightning, porque el modelo y los datos son los mismos.
5. Hay que cambiar la forma de entrada con `unsqueeze(-1)` en PyTorch o con `keras.layers.Reshape` en Keras, y tomar el último estado oculto.
6. Las métricas deberían ser similares, aunque no idénticas, porque la inicialización y el orden de las operaciones de punto flotante difieren. El tiempo depende del *hardware* y del tamaño del modelo.

14. Glosario

Término	Significado
Ventana	Últimas w observaciones que el modelo usa como entrada
Horizonte	Número de pasos futuros que se pronostican
Fuga de información	Uso, al entrenar, de información que no estaría disponible al predecir
Persistencia	Línea base que repite el último valor observado
Estrategia recursiva / directa	Pronóstico paso a paso realimentado / todos los pasos a la vez
`LightningModule`	Clase que agrupa modelo, pasos y optimizador en Lightning
Backend de Keras	Motor de cómputo sobre el que corre Keras 3 (TensorFlow, JAX o PyTorch)

Término	Significado
XLA	Compilador de álgebra lineal que usa JAX para acelerar el cómputo

15. Conclusiones académicas

Qué se aprendió. Pronosticar con redes neuronales exige convertir la serie en un problema supervisado mediante ventanas, y respetar el tiempo en cada decisión: la partición cronológica, la normalización con estadísticas de entrenamiento y el contexto que solo mira al pasado. El mismo modelo se implementó con PyTorch Lightning y con Keras 3 sobre JAX, alimentado por los mismos `DataLoader`.

Qué demostraron los experimentos.

- En prueba, el MAE a un mes fue:

Método	MAE
Persistencia	15.73
Media móvil de 6 meses	13.88
Línea base estacional	31.27
MLP con Lightning	12.49
MLP con Keras/JAX	12.28

  Ambos MLP superaron a las tres líneas base.
- Lightning se detuvo en la época 40 y Keras en la 19, y aun así llegaron a errores parecidos.
- El pronóstico recursivo desde el final de validación tuvo un MAE acumulado de 2.09 en los primeros 12 meses y de 6.40 en 36.
- Promediando sobre los 349 orígenes, la estrategia directa obtuvo MAE 12.16 a un mes y 13.93 a doce, frente a 12.70 y 14.23 de la recursiva.

Cómo interpretar los resultados. La comparación con líneas base es lo que da sentido a las cifras. Un MAE de 12 sin contexto no dice nada; un MAE menor que el de la persistencia muestra que el modelo aporta información. La mala posición de la línea base estacional refleja la irregularidad de los ciclos, no un defecto del método. Que la estrategia directa haya sido mejor en todos los adelantos es un resultado de **esta** serie y de este horizonte, no una regla.

Limitaciones.

- La serie es sintética, univariada y generada con un único proceso.
- Una sola partición temporal, en lugar de una validación con origen móvil (*rolling origin*).
- Una semilla por modelo.
- Solo se estimaron pronósticos puntuales, sin intervalos de predicción.
- No se compararon modelos estadísticos clásicos, como ARIMA o ETS.

Qué conviene profundizar.

- Validación con origen móvil.
- Pronóstico probabilístico (cuantiles o distribuciones).
- Arquitecturas secuenciales: LSTM, convoluciones temporales y *transformers* para series.
- Covariables exógenas.
- Aplicar el mismo *pipeline* a la serie real de manchas solares que usa el notebook de la clase.

Relación con el *Deep Learning*. Las ventanas, los `Dataset` personalizados, los *loggers*, los puntos de control y los programadores de tasa son la base de cualquier modelo secuencial profundo. Y que Keras 3 entrene el mismo modelo sobre JAX con datos servidos por PyTorch muestra que hoy la metodología importa más que el *framework*: los conceptos se trasladan entre bibliotecas casi sin cambios.

16. Referencias

- Hortúa, H. J. *Deep Learning I — Clase 02: `Pipeline__PyTorchL_KerasJAX.ipynb`*. Universidad El Bosque. Referencia conceptual de esta guía (enlaces en el recuadro de procedencia).
- Hyndman, R. J. y Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3.ª ed.). OTexts. https://otexts.com/fpp3/
- Goodfellow, I., Bengio, Y. y Courville, A. (2016). *Deep Learning*. MIT Press. Cap. 10.
- Documentación: Lightning https://lightning.ai/docs/pytorch/stable/ · Keras 3 https://keras.io/ · JAX https://jax.readthedocs.io/